



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación orientada a objetos.

Práctica 9: Clases abstractas e Interfaces.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Itzel Barriba Cázares

Mayo/2026

ÍNDICE

I.	Objetivo	2
II.	Descripción de la practica	2
	Lista de materiales.....	2
III.	Desarrollo de la práctica	3
	Programa 1: Clases Persona y Cliente.....	3
	Programa 2: Clases Cuenta Bancaria y Cuenta de Ahorros.	3
	La clase debe tener los siguientes métodos:	3
	Programa 3: Clases Ship, CruiseShip y CargoShip Diseñe una clase Ship con los siguientes miembros:	4
IV.	Análisis de Resultados	5
	Clase Main	5
	Clase Persona	15
	Clase Cliente	16
	Clase CuentaBancaria	17
	Clase Ship	18
	Clase CargoShip	18
	Clase CruiseShip	19
	Salida en terminal	19
V.	Conclusiones	25
	Uso del constructor this para reutilizar código	25
	Demostración automática como herramienta de verificación.....	25
	Variables fuera del main para simplificar los métodos del menú.....	25
VI.	Bibliografía	25
	Entrega	25
	Criterios para evaluar:	26
	Código (70 puntos).....	26
	Reporte (30 puntos):	26

I. Objetivo

Diseñar clases que modelen entidades del mundo real aplicando correctamente la herencia e integrando el uso de clases abstractas e interfaces para definir jerarquías, comportamientos comunes y favorecer el polimorfismo, utilizando el lenguaje de programación orientada a objetos, con una actitud analítica, propositiva y con creatividad.

II. Descripción de la practica

Lista de materiales

- Computadora Personal (PC)

- Ambiente de desarrollo. (Vscode)
- JDK Java Development Kit

III. Desarrollo de la práctica

Programa 1: Clases Persona y Cliente

Diseñe una clase llamada Persona con campos para almacenar el nombre, la dirección y el número de teléfono. Escriba uno o más constructores y los métodos de acceso y modificación adecuados para los campos de la clase.

A continuación, diseñe una clase llamada Cliente, que herede de la clase Persona. La clase Cliente debe tener un campo para el número de cliente y un campo booleano que indique si el cliente desea estar en una lista de correo. Escriba uno o más constructores y los métodos de acceso y modificación adecuados para los campos de la clase. Muestre un objeto de la clase Cliente en un programa sencillo.

Programa 2: Clases Cuenta Bancaria y Cuenta de Ahorros.

Diseñe una clase abstracta llamada Cuenta Bancaria para almacenar los siguientes datos de una cuenta bancaria:

- Saldo
- Número de depósitos este mes
- Número de retiros
- Tasa de interés anual
- Cargos mensuales por servicio

La clase debe tener los siguientes métodos:

1. Constructor: El constructor debe aceptar argumentos para el saldo y la tasa de interés anual.
2. Depósito: Un método que acepta como argumento el monto del depósito. El método debe sumar el argumento al saldo de la cuenta. También debe incrementar la variable que almacena el número de depósitos.
3. Retirar: Un método que acepta un argumento para el monto del retiro. El método debe restar el argumento del saldo. También debe incrementar la variable que almacena el número de retiros.
4. Calcular Interés: Un método que actualiza el saldo calculando el interés mensual generado por la cuenta y sumándolo al saldo. Esto se realiza mediante las siguientes fórmulas:
 - a. Tasa de Interés Mensual = (Tasa de Interés Anual / 12)
 - b. Interés Mensual = Saldo * Tasa de Interés Mensual
 - c. Saldo = Saldo + Interés Mensual
5. Proceso Mensual: Un método que resta los cargos mensuales por servicio del saldo, llama al método `Calcular Interés` y luego establece a cero las variables que almacenan el número de retiros, el número de depósitos y los cargos mensuales por servicio.
6. A continuación, diseñe una clase `Cuenta de Ahorros` que extienda la clase `CuentaBancaria`. La clase `Cuenta de Ahorros` debe tener un campo `status` para indicar si la cuenta está activa o inactiva. Si el saldo de una cuenta de ahorros cae por debajo de \$25, se vuelve inactiva. (El campo `status` podría ser una variable booleana). No se podrán realizar más retiros hasta que el saldo supere los \$25, momento en el que la cuenta se activará de nuevo.
7. La clase `Cuenta de Ahorros` debe tener los siguientes métodos:
 - a. `Retirar`: Un método que determina si la cuenta está inactiva antes de realizar un retiro. (No se permitirá ningún retiro si la cuenta no está activa). El retiro se realiza llamando a la versión de la superclase del método.
 - b. `Depositar`: Un método que determina si la cuenta está inactiva antes de realizar un depósito. Si la cuenta está inactiva y el depósito eleva el saldo por encima de \$25, la cuenta se activará de nuevo. El depósito se realiza llamando a la versión de la superclase del método.
 - c. Antes de llamar al método de la superclase, este método verifica el número de retiros realizados. Si el número de retiros del mes supera los 4, se añade un cargo por servicio de \$1 por cada retiro adicional al campo de la superclase que almacena los cargos mensuales.

(No olvide verificar el saldo de la cuenta después de aplicar el cargo por servicio. Si el saldo cae por debajo de \$25, la cuenta se desactiva).

Programa 3: Clases Ship, CruiseShip y CargoShip Diseñe una clase Ship con los siguientes miembros:

1. Un campo para el nombre del barco (cadena de texto).
2. Un campo para el año de construcción del barco (cadena de texto).
3. Un constructor y sus correspondientes métodos de acceso y modificación.
4. Un método toString que muestre el nombre del barco y el año de construcción.

Diseñe una clase CruiseShip que extienda la clase Ship.

La clase CruiseShip debe tener los siguientes miembros:

- Un campo para el número máximo de pasajeros (entero).
- Un constructor y sus correspondientes métodos de acceso y modificación.
- Un método `toString` que sobrescribe el método `toString` de la clase base. El método `toString` de la clase `CruiseShip` debe mostrar únicamente el nombre del barco y el número máximo de pasajeros. Diseñe una clase `CargoShip` que extienda la clase `Ship`. La clase `CargoShip` debe tener los siguientes miembros:
 - Un campo para la capacidad de carga en toneladas (un entero).
 - Un constructor y sus correspondientes métodos de acceso y modificación.
 - Un método `toString` que sobrescribe el método `toString` de la clase base. El método `toString` de la clase `CargoShip` debe mostrar únicamente el nombre del barco y su capacidad de carga.
- Demostremos el funcionamiento de las clases en un programa con un array de `Ship`. Asigne varios objetos de tipo `Ship`, `CruiseShip` y `CargoShip` a los elementos del array. El programa debe recorrer el array, llamando al método `toString` de cada objeto.

IV. Análisis de Resultados

Clase Main

```
import java.util.Scanner;

public class Main {
    private static Scanner input = new Scanner(System.in);
    private static Cliente clienteActual = new Cliente();
    private static CuentaAhorros cuentaActual = null;
    private static boolean cuentaCreada = false;
    private static Ship[] barcos = new Ship[10];
    private static int contadorBarcos = 0;

    public static void main(String[] args) {
        int opcion;
        do {
            System.out.println("\n=====");
            System.out.println("                MENÚ PRINCIPAL                ");
            System.out.println("=====");
            System.out.println("\t1 - Probar Programa 1: Cliente");
            System.out.println("\t2 - Probar Programa 2: Cuenta de Ahorros");
            System.out.println("\t3 - Probar Programa 3: Barcos ");
            System.out.println("\t4 - Demostración automática completa");
            System.out.println("\t5 - Salir");
            System.out.print("\n\tOpción: ");
            opcion = input.nextInt();
            input.nextLine();
            switch (opcion) {
                case 1:
                    programaCliente();
                    break;
                case 2:
                    programaCuentaAhorros();
                    break;
                case 3:
                    programaBarcos();
                    break;
                case 4:
                    demostracionAutomatica();
                    break;
                case 5:
                    System.out.println("\nSaliendo");
                    break;
                default:
                    System.out.println("Opción no válida.");
            }
        } while (opcion != 5);
        input.close();
    }
}
```

```
/* ===== PROGRAMA 1: CLIENTE ===== */
private static void programaCliente() {
    int opcion;
    do {
        System.out.println("\n--- PROGRAMA 1: CLIENTE ---");
        System.out.println("\t1 - Crear nuevo cliente");
        System.out.println("\t2 - Modificar datos del cliente");
        System.out.println("\t3 - Mostrar información del cliente");
        System.out.println("\t4 - Volver al menú principal");
        System.out.print("\n\tOpción: ");
        opcion = input.nextInt();
        input.nextLine();
        switch (opcion) {
            case 1:
                crearCliente();
                break;
            case 2:
                modificarCliente();
                break;
            case 3:
                mostrarCliente();
                break;
            case 4:
                System.out.println("\nVolviendo al menú principal...");
                break;
            default:
                System.out.println("Opción no válida.");
        }
    } while (opcion != 4);
}
```

```

private static void crearCliente() {
    System.out.println("\n--- CREAR CLIENTE ---");
    System.out.print("Nombre: ");
    String nombre = input.nextLine();
    System.out.print("Dirección: ");
    String direccion = input.nextLine();
    System.out.print("Teléfono: ");
    String telefono = input.nextLine();
    System.out.print("Número de cliente: ");
    int numCliente = input.nextInt();
    input.nextLine();
    System.out.print("¿Desea estar en lista de correo? (s/n): ");
    String respuesta = input.nextLine();
    boolean listaCorreo = respuesta.equalsIgnoreCase("s");

    clienteActual = new Cliente(nombre, direccion, telefono, numCliente, listaCorreo);
    System.out.println("\nCliente creado exitosamente.");
}

private static void modificarCliente() {
    System.out.println("\n--- MODIFICAR CLIENTE ---");
    System.out.print("Nuevo nombre: ");
    clienteActual.setNombre(input.nextLine());
    System.out.print("Nueva dirección: ");
    clienteActual.setDireccion(input.nextLine());
    System.out.print("Nuevo teléfono: ");
    clienteActual.setTelefono(input.nextLine());
    System.out.print("Nuevo número de cliente: ");
    clienteActual.setNumeroCliente(input.nextInt());
    input.nextLine();
    System.out.print("¿Desea estar en lista de correo? (s/n): ");
    String respuesta = input.nextLine();
    boolean listaCorreo = respuesta.equalsIgnoreCase("s");
    System.out.println("\nCliente modificado exitosamente.");
}

private static void mostrarCliente() {
    System.out.println("\n--- INFORMACIÓN DEL CLIENTE ---");
    System.out.println(clienteActual);
}

```

```
/* ===== PROGRAMA 2: CUENTA DE AHORROS ===== */
```

```
private static void programaCuentaAhorros() {
    int opcion;
    do {
        System.out.println("\n--- PROGRAMA 2: CUENTA DE AHORROS ---");
        System.out.println("\t1 - Crear cuenta de ahorros");
        System.out.println("\t2 - Realizar depósito");
        System.out.println("\t3 - Realizar retiro");
        System.out.println("\t4 - Mostrar información de la cuenta");
        System.out.println("\t5 - Procesar mes");
        System.out.println("\t6 - Volver al menú principal");
        System.out.print("\n\tOpción: ");
        opcion = input.nextInt();
        input.nextLine();
        switch (opcion) {
            case 1:
                crearCuenta();
                break;
            case 2:
                realizarDeposito();
                break;
            case 3:
                realizarRetiro();
                break;
            case 4:
                mostrarInformacionCuenta();
                break;
            case 5:
                procesarMes();
                break;
            case 6:
                System.out.println("\nVolviendo al menú principal...");
                break;
            default:
                System.out.println("Opción no válida.");
        }
    } while (opcion != 6);
}
```

```
private static void crearCuenta() {
    System.out.println("\n--- CREAR CUENTA ---");
    System.out.print("Saldo inicial: $");
    double saldo = input.nextDouble();
    System.out.print("Tasa de interés anual (ej: 0.05 = 5%): ");
    double tasa = input.nextDouble();
    input.nextLine();

    cuentaActual = new CuentaAhorros(saldo, tasa);
    cuentaCreada = true;
    System.out.printf("\nCuenta creada exitosamente. Saldo: $%.2f\n", cuentaActual.getSaldo());
    System.out.printf(" Estado: %s\n", cuentaActual.getSaldo() >= 25 ? "Activa" : "Inactiva");
}
```



```

private static void realizarDeposito() {
    if (!validarCuenta())
        return;
    System.out.print("\nMonto a depositar: $");
    double monto = input.nextDouble();
    input.nextLine();

    double saldoAntes = cuentaActual.getSaldo();
    cuentaActual.depositar(monto);
    System.out.printf("Depósito realizado. Saldo anterior: $%.2f → Saldo actual: $%.2f\n",
        saldoAntes, cuentaActual.getSaldo());
    System.out.printf(" Estado de la cuenta: %s\n", cuentaActual.getSaldo() >= 25 ? "Activa"
: "Inactiva");
}

private static void realizarRetiro() {
    if (!validarCuenta())
        return;
    System.out.print("\nMonto a retirar: $");
    double monto = input.nextDouble();
    input.nextLine();

    double saldoAntes = cuentaActual.getSaldo();
    cuentaActual.retirar(monto);
    if (saldoAntes != cuentaActual.getSaldo()) {
        System.out.printf("Retiro realizado. Saldo anterior: $%.2f → Saldo actual: $%.2f\n",
            saldoAntes, cuentaActual.getSaldo());
        System.out.printf(" Retiros realizados este mes: %d\n", cuentaActual.getNumReti-
ros());
    }
}

private static void mostrarInformacionCuenta() {
    if (!validarCuenta()) {
        return;
    }
    System.out.println("\n--- INFORMACIÓN DE LA CUENTA ---");
    System.out.printf("Saldo actual: $%.2f\n", cuentaActual.getSaldo());
    System.out.printf("Estado: %s\n", cuentaActual.getSaldo() >= 25 ? "ACTIVA" : "INACTIVA");
    System.out.printf("Depósitos este mes: %d\n", cuentaActual.getNumDepositos());
    System.out.printf("Retiros este mes: %d\n", cuentaActual.getNumRetiros());
    System.out.printf("Cargos mensuales acumulados: $%.2f\n", cuentaActual.getCargosMensua-
les());
    System.out.printf("Tasa de interés anual: %.2f%%\n", cuentaActual.getTasaInteresAnual() *
100);
}

```

```

private static void procesarMes() {
    if (!validarCuenta()) {
        return;
    }
    System.out.println("\n--- PROCESANDO MES ---");
    System.out.printf("Saldo antes del proceso: $%.2f\n", cuentaActual.getSaldo());
    cuentaActual.procesoMensual();
    System.out.printf("Mes procesado. Saldo después de interés y cargos: $%.2f\n", cuentaActual.getSaldo());
    System.out.printf("  Depósitos reiniciados: %d\n", cuentaActual.getNumDepositos());
    System.out.printf("  Retiros reiniciados: %d\n", cuentaActual.getNumRetiros());
    System.out.printf("  Cargos reiniciados: $%.2f\n", cuentaActual.getCargosMensuales());
}

/* ===== PROGRAMA 3: BARCOS ===== */

private static void programaBarcos() {
    int opcion;
    do {
        System.out.println("\n--- PROGRAMA 3: BARCOS (POLIMORFISMO) ---");
        System.out.println("\t1 - Agregar barco genérico");
        System.out.println("\t2 - Agregar crucero (CruiseShip)");
        System.out.println("\t3 - Agregar barco de carga (CargoShip)");
        System.out.println("\t4 - Mostrar todos los barcos");
        System.out.println("\t5 - Mostrar información específica de un barco");
        System.out.println("\t6 - Volver al menú principal");
        System.out.print("\n\tOpción: ");
        opcion = input.nextInt();
        input.nextLine();
        switch (opcion) {
            case 1:
                agregarBarcoGenerico();
                break;
            case 2:
                agregarCrucero();
                break;
            case 3:
                agregarCarguero();
                break;
            case 4:
                mostrarTodosLosBarcos();
                break;
            case 5:
                mostrarInformacionBarco();
                break;
            case 6:
                System.out.println("\nVolviendo al menú principal...");
                break;
            default:
                System.out.println("Opción no válida.");
        }
    } while (opcion != 6);
}

```

```

private static void agregarBarcoGenerico() {
    if (!validarCapacidadBarcos())
        return;
    System.out.println("\n--- AGREGAR BARCO ---");
    System.out.print("Nombre: ");
    String nombre = input.nextLine();
    System.out.print("Año de construcción: ");
    String año = input.nextLine();
    barcos[contadorBarcos++] = new Ship(nombre, año);
    System.out.println("Barco agregado exitosamente.");
}

private static void agregarCrucero() {
    if (!validarCapacidadBarcos())
        return;
    System.out.println("\n--- AGREGAR CRUCERO ---");
    System.out.print("Nombre: ");
    String nombre = input.nextLine();
    System.out.print("Año de construcción: ");
    String año = input.nextLine();
    System.out.print("Número máximo de pasajeros: ");
    int maxPasajeros = input.nextInt();
    input.nextLine();
    barcos[contadorBarcos++] = new CruiseShip(nombre, año, maxPasajeros);
    System.out.println("Crucero agregado exitosamente.");
}

private static void agregarCarguero() {
    if (!validarCapacidadBarcos())
        return;
    System.out.println("\n--- AGREGAR BARCO DE CARGA ---");
    System.out.print("Nombre: ");
    String nombre = input.nextLine();
    System.out.print("Año de construcción: ");
    String año = input.nextLine();
    System.out.print("Capacidad de carga (toneladas): ");
    int capacidad = input.nextInt();
    input.nextLine();
    barcos[contadorBarcos++] = new CargoShip(nombre, año, capacidad);
    System.out.println("Barco de carga agregado exitosamente.");
}

private static void mostrarTodosLosBarcos() {
    if (contadorBarcos == 0) {
        System.out.println("\nNo hay barcos registrados.");
        return;
    }
    System.out.println("\n--- LISTA DE BARCOS ---");
    System.out.println("=====");
    for (int i = 0; i < contadorBarcos; i++) {
        System.out.printf("[%d] %s\n", i + 1, barcos[i].toString());
    }
}

```

```

private static void mostrarInformacionBarco() {
    if (contadorBarcos == 0) {
        System.out.println("\nNo hay barcos registrados.");
        return;
    }

    System.out.println("\n--- SELECCIONE UN BARCO ---");
    for (int i = 0; i < contadorBarcos; i++) {
        System.out.printf("[%d] %s\n", i + 1, barcos[i].getNombre());
    }
    System.out.print("\nOpción: ");
    int seleccion = input.nextInt();
    input.nextLine();

    if (seleccion >= 1 && seleccion <= contadorBarcos) {
        imprimirDetalleBarco(seleccion - 1);
    } else {
        System.out.println("Selección no válida.");
    }
}

private static void imprimirDetalleBarco(int indice) {
    System.out.println("\n--- INFORMACIÓN DEL BARCO ---");
    System.out.println(barcos[indice].toString());

    if (barcos[indice] instanceof CruiseShip) {
        CruiseShip cs = (CruiseShip) barcos[indice];
        System.out.println(" Tipo: Crucero");
        System.out.println(" Año de construcción: " + cs.getAñoConstruccion());
    } else if (barcos[indice] instanceof CargoShip) {
        CargoShip cgs = (CargoShip) barcos[indice];
        System.out.println(" Tipo: Barco de carga");
        System.out.println(" Año de construcción: " + cgs.getAñoConstruccion());
    } else {
        System.out.println(" Tipo: Barco genérico");
    }
}

/* ===== DEMOSTRACIÓN AUTOMÁTICA ===== */

private static void demostracionAutomatica() {
    System.out.println("\n=====");
    System.out.println(" DEMOSTRACIÓN AUTOMÁTICA COMPLETA ");
    System.out.println("===== \n");

    demostrarPrograma1();
    demostrarPrograma2();
    demostrarPrograma3();

    System.out.println("\n=====");
    System.out.println(" DEMOSTRACIÓN COMPLETADA ");
    System.out.println("===== \n");
}

```

```

private static void demostrarPrograma1() {
    System.out.println(">>> PROGRAMA 1: CLIENTE (Herencia)");
    System.out.println("-----");
    Cliente clienteDemo = new Cliente("María Pérez", "Calle Falsa 123", "555-9999", 2001,
true);
    System.out.println("Cliente creado:");
    System.out.println("  " + clienteDemo);
    System.out.println("\nModificando datos del cliente...");
    clienteDemo.setDireccion("Avenida Siempre Viva 742");
    clienteDemo.setTelefono("555-8888");
    System.out.println("  " + clienteDemo);
}

private static void demostrarPrograma2() {
    System.out.println("\n\n>>> PROGRAMA 2: CUENTA DE AHORROS");
    System.out.println("-----");
    CuentaAhorros cuentaDemo = new CuentaAhorros(100, 0.05);
    System.out.println("Cuenta creada con saldo inicial: $100");

    System.out.println("\nRealizando depósitos y retiros:");
    cuentaDemo.depositar(50);
    System.out.printf(" Depósito de $50 → Saldo: $%.2f (Activa: %s)\n",
        cuentaDemo.getSaldo(), cuentaDemo.getSaldo() >= 25 ? "Sí" : "No");

    cuentaDemo.retirar(30);
    System.out.printf(" Retiro de $30 → Saldo: $%.2f (Activa: %s)\n",
        cuentaDemo.getSaldo(), cuentaDemo.getSaldo() >= 25 ? "Sí" : "No");

    cuentaDemo.retirar(100);
    System.out.printf(" Intento de retiro de $100 → Saldo: $%.2f\n", cuentaDemo.getSaldo());

    System.out.println("\nProbando límite de retiros (más de 4):");
    CuentaAhorros cuentaRetiros = new CuentaAhorros(500, 0.05);
    for (int i = 1; i <= 5; i++) {
        cuentaRetiros.retirar(10);
        System.out.printf(" Retiro #%d → Saldo: $%.2f, Retiros: %d, Cargos: $%.2f\n",
            i, cuentaRetiros.getSaldo(),
            cuentaRetiros.getNumRetiros(),
            cuentaRetiros.getCargosMensuales());
    }

    System.out.println("\nProcesando fin de mes...");
    cuentaRetiros.procesoMensual();
    System.out.printf(" Después del proceso → Saldo: $%.2f, Retiros: %d, Cargos: $%.2f\n",
        cuentaRetiros.getSaldo(), cuentaRetiros.getNumRetiros(),
        cuentaRetiros.getCargosMensuales());
}

```

```

private static void demostrarPrograma3() {
    System.out.println("\n\n>>> PROGRAMA 3: BARCOS (Polimorfismo)");
    System.out.println("-----");
    Ship[] barcosDemo = new Ship[5];

    barcosDemo[0] = new Ship("Titanic", "1912");
    barcosDemo[1] = new CruiseShip("Royal Caribbean", "2015", 6000);
    barcosDemo[2] = new CargoShip("MSC Christina", "2008", 85000);
    barcosDemo[3] = new CruiseShip("Disney Wonder", "1999", 2400);
    barcosDemo[4] = new CargoShip("Ever Given", "2018", 200000);

    System.out.println("Array de barcos (demostración de polimorfismo):\n");
    for (int i = 0; i < barcosDemo.length; i++) {
        System.out.printf(" [%d] %s\n", i + 1, barcosDemo[i].toString());
    }
}

/* ===== METODOS AUXILIARES ===== */

private static boolean validarCuenta() {
    if (!cuentaCreada) {
        System.out.println("\nError: Primero debe crear una cuenta (Opción 1).");
        return false;
    }
    return true;
}

private static boolean validarCapacidadBarcos() {
    if (contadorBarcos >= barcos.length) {
        System.out.println("Límite de barcos alcanzado.");
        return false;
    }
    return true;
}
}

```

Clase Persona

```
public class Persona {
    private String nombre;
    private String direccion;
    private String telefono;

    // Constructores
    public Persona() {
        this.nombre = "";
        this.direccion = "";
        this.telefono = "";
    }

    public Persona(String nombre, String direccion, String telefono) {
        this.nombre = nombre;
        this.direccion = direccion;
        this.telefono = telefono;
    }

    // Métodos de acceso
    public String getNombre() {
        return nombre;
    }

    public String getDireccion() {
        return direccion;
    }

    public String getTelefono() {
        return telefono;
    }

    // Métodos de modificación
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public void setDireccion(String direccion) {
        this.direccion = direccion;
    }

    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }

    public String toString() {
        return "Nombre: " + nombre + ", Dirección: " + direccion + ", Teléfono: " + telefono;
    }
}
```

Clase Cliente

```

class Cliente extends Persona {
    private int numeroCliente;
    private boolean listaCorreo;

    // Constructores
    public Cliente() {
        super();
        this.numeroCliente = 0;
        this.listaCorreo = false;
    }

    public Cliente(String nombre, String direccion, String telefono, int numeroCliente, boolean
listaCorreo) {
        super(nombre, direccion, telefono);
        this.numeroCliente = numeroCliente;
        this.listaCorreo = listaCorreo;
    }

    // Métodos de acceso
    public int getNumeroCliente() {
        return numeroCliente;
    }

    public boolean isListaCorreo() {
        return listaCorreo;
    }

    // Métodos de modificación
    public void setNumeroCliente(int numeroCliente) {
        this.numeroCliente = numeroCliente;
    }

    public void setListaCorreo(boolean listaCorreo) {
        this.listaCorreo = listaCorreo;
    }

    @Override
    public String toString() {
        return super.toString() + ", Número Cliente: " + numeroCliente + ", En lista de correo: "
+ (listaCorreo ? "Sí" : "No");
    }
}

```


Clase CuentaBancaria

```
class CuentaAhorros extends CuentaBancaria {
    private boolean activa;

    public CuentaAhorros(double saldo, double tasaInteresAnual) {
        super(saldo, tasaInteresAnual);
        this.activa = saldo >= 25;
    }

    private void verificarActividad() {
        activa = saldo >= 25;
    }

    @Override
    public void retirar(double monto) {
        if (!activa) {
            System.out.println("Cuenta inactiva. No se puede realizar el retiro.");
            return;
        }

        if (numRetiros >= 4) {
            cargosMensuales += 1;
            System.out.println("Cargo por servicio de $1 aplicado por retiro adicional.");
        }

        super.retirar(monto);
        verificarActividad();
    }

    @Override
    public void depositar(double monto) {
        super.depositar(monto);
        verificarActividad();
    }
}
```

Clase Ship

```
class Ship {
    protected String nombre;
    protected String añoConstruccion;

    public Ship(String nombre, String añoConstruccion) {
        this.nombre = nombre;
        this.añoConstruccion = añoConstruccion;
    }

    public String getNombre() {
        return nombre;
    }

    public String getAñoConstruccion() {
        return añoConstruccion;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public void setAñoConstruccion(String añoConstruccion) {
        this.añoConstruccion = añoConstruccion;
    }

    public String toString() {
        return "Nombre del barco: " + nombre + ", Año de construcción: " + añoConstruccion;
    }
}
```

Clase CargoShip

```
class CargoShip extends Ship {
    private int capacidadCarga;

    public CargoShip(String nombre, String añoConstruccion, int capacidadCarga) {
        super(nombre, añoConstruccion);
        this.capacidadCarga = capacidadCarga;
    }

    public int getCapacidadCarga() {
        return capacidadCarga;
    }

    public void setCapacidadCarga(int capacidadCarga) {
        this.capacidadCarga = capacidadCarga;
    }

    @Override
    public String toString() {
        return "Nombre del barco: " + nombre + ", Capacidad de carga: " + capacidadCarga + " toneladas";
    }
}
```

Clase CruiseShip

```
class CruiseShip extends Ship {
    private int maxPasajeros;

    public CruiseShip(String nombre, String añoConstruccion, int maxPasajeros) {
        super(nombre, añoConstruccion);
        this.maxPasajeros = maxPasajeros;
    }

    public int getMaxPasajeros() {
        return maxPasajeros;
    }

    public void setMaxPasajeros(int maxPasajeros) {
        this.maxPasajeros = maxPasajeros;
    }

    @Override
    public String toString() {
        return "Nombre del barco: " + nombre + ", Máximo de pasajeros: " + maxPasajeros;
    }
}
```

Salida en terminal

```
> javac -d output -sourcepath src src/*.java
> java -cp output Main
```

MENÚ PRINCIPAL

```

1 - Probar Programa 1: Cliente
2 - Probar Programa 2: Cuenta de Ahorros
3 - Probar Programa 3: Barcos
4 - Demostración automática completa
5 - Salir

Opción: |
```

```

MENÚ PRINCIPAL
1 - Probar Programa 1: Cliente
2 - Probar Programa 2: Cuenta de Ahorros
3 - Probar Programa 3: Barcos
4 - Demostración automática completa
5 - Salir

Opción: 1

— PROGRAMA 1: CLIENTE —
1 - Crear nuevo cliente
2 - Modificar datos del cliente
3 - Mostrar información del cliente
4 - Volver al menú principal

Opción: 1

— CREAR CLIENTE —
Nombre: Joe osorio
Dirección: Universidad 14418, UABC, Parque Internacional Industrial
Teléfono: 6649797500
Número de cliente: 420
¿Desea estar en lista de correo? (s/n): s

Cliente creado exitosamente.

— PROGRAMA 1: CLIENTE —
1 - Crear nuevo cliente
2 - Modificar datos del cliente
3 - Mostrar información del cliente
4 - Volver al menú principal

— MODIFICAR CLIENTE —
Nuevo nombre: joe osorio
Nueva dirección: Universidad 14418, UABC, Parque Internacional Industrial
Nuevo teléfono: 6649797500
Nuevo número de cliente: 469
¿Desea estar en lista de correo? (s/n): n

Cliente modificado exitosamente.

— PROGRAMA 1: CLIENTE —
1 - Crear nuevo cliente
2 - Modificar datos del cliente
3 - Mostrar información del cliente
4 - Volver al menú principal

Opción: 3

— INFORMACIÓN DEL CLIENTE —
Nombre: joe osorio, Dirección: Universidad 14418, UABC, Parque Internacional Industrial, Teléfono: 6649797500, Número Cliente: 469, En lista de correo: Sí

— PROGRAMA 1: CLIENTE —
1 - Crear nuevo cliente
2 - Modificar datos del cliente
3 - Mostrar información del cliente
4 - Volver al menú principal

Opción: 4

Volviendo al menú principal...

```

```
~\OneDrive - uabc.edu.mx\8. X + v - □ X

Opción: 4

Volviendo al menú principal...

=====
                        MENÚ PRINCIPAL
=====

1 - Probar Programa 1: Cliente
2 - Probar Programa 2: Cuenta de Ahorros
3 - Probar Programa 3: Barcos
4 - Demostración automática completa
5 - Salir

Opción: 2

— PROGRAMA 2: CUENTA DE AHORROS —
1 - Crear cuenta de ahorros
2 - Realizar depósito
3 - Realizar retiro
4 - Mostrar información de la cuenta
5 - Procesar mes
6 - Volver al menú principal

Opción: 1

— CREAR CUENTA —
Saldo inicial: $1500
Tasa de interés anual (ej: 0.05 = 5%): 0.03

Cuenta creada exitosamente. Saldo: $1500.00
Estado: Activa

— PROGRAMA 2: CUENTA DE AHORROS —
1 - Crear cuenta de ahorros
2 - Realizar depósito
3 - Realizar retiro
4 - Mostrar información de la cuenta
5 - Procesar mes
6 - Volver al menú principal

Opción: 2

Monto a depositar: $420
Depósito realizado. Saldo anterior: $1500.00 ? Saldo actual: $1920.00
Estado de la cuenta: Activa

— PROGRAMA 2: CUENTA DE AHORROS —
1 - Crear cuenta de ahorros
2 - Realizar depósito
3 - Realizar retiro
4 - Mostrar información de la cuenta
5 - Procesar mes
6 - Volver al menú principal

Opción: 3

Monto a retirar: $404
Retiro realizado. Saldo anterior: $1920.00 ? Saldo actual: $1516.00
Retiros realizados este mes: 1
```

```

— PROGRAMA 2: CUENTA DE AHORROS —
  1 - Crear cuenta de ahorros
  2 - Realizar depósito
  3 - Realizar retiro
  4 - Mostrar información de la cuenta
  5 - Procesar mes
  6 - Volver al menú principal

    Opción: 4

— INFORMACIÓN DE LA CUENTA —
Saldo actual: $1516.00
Estado: ACTIVA
Depósitos este mes: 1
Retiros este mes: 1
Cargos mensuales acumulados: $0.00
Tasa de interés anual: 3.00%

— PROGRAMA 2: CUENTA DE AHORROS —
  1 - Crear cuenta de ahorros
  2 - Realizar depósito
  3 - Realizar retiro
  4 - Mostrar información de la cuenta
  5 - Procesar mes
  6 - Volver al menú principal

    Opción: 5

— PROCESANDO MES —
Saldo antes del proceso: $1516.00
Mes procesado. Saldo después de interés y cargos: $1519.79
  Depósitos reiniciados: 0
  Retiros reiniciados: 0
  Cargos reiniciados: $0.00

— PROGRAMA 2: CUENTA DE AHORROS —
  1 - Crear cuenta de ahorros
  2 - Realizar depósito
  3 - Realizar retiro
  4 - Mostrar información de la cuenta
  5 - Procesar mes
  6 - Volver al menú principal

    Opción: 4

— INFORMACIÓN DE LA CUENTA —
Saldo actual: $1519.79
Estado: ACTIVA
Depósitos este mes: 0
Retiros este mes: 0
Cargos mensuales acumulados: $0.00
Tasa de interés anual: 3.00%

— PROGRAMA 2: CUENTA DE AHORROS —
  1 - Crear cuenta de ahorros
  2 - Realizar depósito
  3 - Realizar retiro
  4 - Mostrar información de la cuenta
  5 - Procesar mes
  6 - Volver al menú principal

    Opción: |
  
```

```

~\OneDrive - uabc.edu.mx\8  X + v - □ X

1 - Probar Programa 1: Cliente
2 - Probar Programa 2: Cuenta de Ahorros
3 - Probar Programa 3: Barcos
4 - Demostración automática completa
5 - Salir

Opción: 3

— PROGRAMA 3: BARCOS (POLIMORFISMO) —
1 - Agregar barco genérico
2 - Agregar crucero (CruiseShip)
3 - Agregar barco de carga (CargoShip)
4 - Mostrar todos los barcos
5 - Mostrar información específica de un barco
6 - Volver al menú principal

Opción: 1

— AGREGAR BARCO —
Nombre: La niña
Año de construcción: 1992
Barco agregado exitosamente.

— PROGRAMA 3: BARCOS (POLIMORFISMO) —
1 - Agregar barco genérico
2 - Agregar crucero (CruiseShip)
3 - Agregar barco de carga (CargoShip)
4 - Mostrar todos los barcos
5 - Mostrar información específica de un barco
6 - Volver al menú principal

Opción: 2

— AGREGAR CRUCERO —
Nombre: Santa maria
Año de construcción: 1491
Número máximo de pasajeros: 100
Crucero agregado exitosamente.

— PROGRAMA 3: BARCOS (POLIMORFISMO) —
1 - Agregar barco genérico
2 - Agregar crucero (CruiseShip)
3 - Agregar barco de carga (CargoShip)
4 - Mostrar todos los barcos
5 - Mostrar información específica de un barco
6 - Volver al menú principal

Opción: 3

— AGREGAR BARCO DE CARGA —
Nombre: La pinta
Año de construcción: 1519
Capacidad de carga (toneladas): 1500
Barco de carga agregado exitosamente.

— PROGRAMA 3: BARCOS (POLIMORFISMO) —
1 - Agregar barco genérico
2 - Agregar crucero (CruiseShip)
3 - Agregar barco de carga (CargoShip)
4 - Mostrar todos los barcos
5 - Mostrar información específica de un barco
6 - Volver al menú principal

Opción: 4

```

— LISTA DE BARCOS —

-
-
- [1] Nombre del barco: La niña, Año de construcción: 1992
 - [2] Nombre del barco: Santa maria, Máximo de pasajeros: 100
 - [3] Nombre del barco: La pinta, Capacidad de carga: 1500 toneladas

— PROGRAMA 3: BARCOS (POLIMORFISMO) —

- 1 - Agregar barco genérico
- 2 - Agregar crucero (CruiseShip)
- 3 - Agregar barco de carga (CargoShip)
- 4 - Mostrar todos los barcos
- 5 - Mostrar información específica de un barco
- 6 - Volver al menú principal

Opción: 5

— SELECCIONE UN BARCO —

- [1] La niña
- [2] Santa maria
- [3] La pinta

Opción: 2

— INFORMACIÓN DEL BARCO —

Nombre del barco: Santa maria, Máximo de pasajeros: 100
 Tipo: Crucero
 Año de construcción: 1491

— PROGRAMA 3: BARCOS (POLIMORFISMO) —

- 1 - Agregar barco genérico
- 2 - Agregar crucero (CruiseShip)
- 3 - Agregar barco de carga (CargoShip)
- 4 - Mostrar todos los barcos
- 5 - Mostrar información específica de un barco
- 6 - Volver al menú principal

Opción: 6

Volviendo al menú principal...

 MENÚ PRINCIPAL

- 1 - Probar Programa 1: Cliente
- 2 - Probar Programa 2: Cuenta de Ahorros
- 3 - Probar Programa 3: Barcos
- 4 - Demostración automática completa
- 5 - Salir

Opción: |

V. Conclusiones

Uso del constructor this para reutilizar código

La implementación del constructor this permitió invocar un constructor desde otro dentro de la misma clase, evitando repetir la inicialización de atributos comunes. Por ejemplo, en las clases Libro y Persona, los constructores secundarios llaman al principal usando "this()", lo que reduce código redundante y facilita el mantenimiento.

Demostración automática como herramienta de verificación

La inclusión de una opción de "Demostración automática completa" (opción 4 en el menú) permitió probar todas las funcionalidades de los tres programas de una sola vez. Esto evitó tener que ejecutar manualmente cada opción del menú repetidamente para validar el correcto funcionamiento de los métodos, agilizando la fase de pruebas y depuración.

Variables fuera del main para simplificar los métodos del menú

Declarar variables como clienteActual, cuentaActual, barcos[] y contadorBarcos como atributos estáticos de la clase principal (fuera del método main) facilitó su acceso y modificación desde cualquier método del menú. De lo contrario, habría sido necesario pasar estos objetos como parámetros constantemente, aumentando la complejidad del código. Esta organización hizo que los métodos fueran más cortos, legibles y fáciles de implementar.

VI. Bibliografía

- [1] R. Monteagudo, "¿Para que sirve el metodo this en Java?," Stack Overflow En Español, May 26, 2020. Available: <https://es.stackoverflow.com/questions/359264/para-que-sirve-el-metodo-this-en-java>
- [2] PrgmRNoob, "Creating a console menu for user to make a selection," Stack Overflow, Apr. 20, 2024. Available: <https://stackoverflow.com/questions/8059259/creating-a-console-menu-for-user-to-make-a-selection/79940305#79940305>
- [3] Stepanenko, "Cómo crear un menú en consola con el lenguaje de programación Java," Masterhacks Blog, Sep. 03, 2024. Available: <https://blogs.masterhacks.net/programacion/como-crear-un-menu-en-consola-con-el-lenguaje-de-programacion-java/>
- [4] C. Á. Caules, "Java Constructor y buenas prácticas," Arquitectura Java, Jan. 08, 2022. Available: <https://www.arquitecturajava.com/java-constructor-y-buenas-practicas/>
- [5] A. Duggal, "How to Use Constructors in Java: A Beginner's Guide," freeCodeCamp.org, Jul. 08, 2025. Available: <https://www.freecodecamp.org/news/how-to-use-constructors-in-java-a-beginners-guide/>
- [6] Peter, "Best way to handle multiple constructors in Java," Stack Overflow, Feb. 24, 2009. Available: <https://stackoverflow.com/questions/581873/best-way-to-handle-multiple-constructors-in-java>
- [7] A. Divertitto, "Java extiende la palabra clave con ejemplos," CodeGym, Feb. 01, 2024. Available: <https://codegym.cc/es/groups/posts/es.668.java-extiende-la-palabra-clave-con-ejemplos>
- [8] User, "Mostrar los datos con formato correcto en Java," Stack Overflow En Español, Mar. 13, 2024. Available: <https://es.stackoverflow.com/questions/616276/mostrar-los-datos-con-formato-correcto-en-java>
- [9] "Formato de Salida en Java con printf | Blog Coders Free," Coders Free. Available: <https://codersfree.com/posts/formato-de-salida-en-java-con-printf>

Entrega

- La entrega se realizará mediante una actividad en BB. Se han de entregar los archivos fuentes desarrollados, más el reporte de la práctica adjuntos adjuntos en un fichero zip cuyo nombre esté formado por el número de práctica y nombre del alumno. Ejemplo Practica4JuanLopez.pdf
- Hacer una clase por cada ejercicio y adjuntarlos

- Entregar un reporte de práctica, formato reporte técnico y electrónico, podrá utilizar este formato y agregar el procedimiento correspondiente), que conceptos fundamentales se aplicaron en esta práctica, junto con resultados y conclusiones.

Criterios para evaluar:

Código (70 puntos)

- Debe incluir comentarios en su programa, sin llegar al uso exagerado de ellos.
- Debe utilizar sangrías en los lugares apropiados.
- Poner como encabezado del programa en comentarios, título, la descripción de la práctica, nombre del alumno que realizó la práctica.
- Funcionamiento correcto del programa

Reporte (30 puntos):

Portada, competencia u objetivo	2.5 puntos
Pegar Código del programa (anexar archivos .py)	2.5 puntos
Explicación completa del código y su funcionamiento	15 puntos
Resultados (análisis de resultados)	5 puntos
Conclusiones:	5 puntos

Nota: No se aceptarán trabajos fuera de la fecha de entrega o que no cumplan con las especificaciones de entrega requeridas. Los trabajos que se detecten copiados serán anulados automáticamente para ambos alumnos.